



mykilOS 6

Datenflüsse, Integrationen & Funktionsbaum

Vollständige, aus dem Quellcode verifizierte Übersicht: welche externen Dienste wo und wie mit mykilOS 6 kommunizieren, mit welchen Protokollen, Triggern, Handshakes und Referenz-Handles — plus das interne Signal-Nervensystem, die Persistenz und der gesamte Funktionsbaum der App. **local-first**, macOS 14+, SwiftUI. Secrets ausschließlich in der macOS-Keychain.

6 Live-Integrationen

Google Workspace · Clockodo · Airtable · ClickUp · Sevdesk · Anthropic Claude. Alle Geschäftsdaten werden **read-only** gelesen; einzig Claude ist ein ausgehender Request/Response-Dienst.

Alle Widgets live

Geld (Tiefblau) liest jetzt live aus **Sevdesk**: Ist-Umsatz (Summe der Rechnungen für sevdeskRef) gegen das Soll-Budget aus Airtable. Kein Stub-Widget mehr.

1 geplanter Anschluss

Drive-Webhook als Live-Quelle für offerDetected — heute feuert das Signal noch per Demo-Button. Alle anderen Handles sind verdrahtet.

Legende

- Drive / Dateien

Menschen / Kalender

Aufgaben (ClickUp)

Geld (Sevdesk)

Notizen / Mail

Assistent / Claude
- LIVE

 echte API, im Einsatz
- STUB

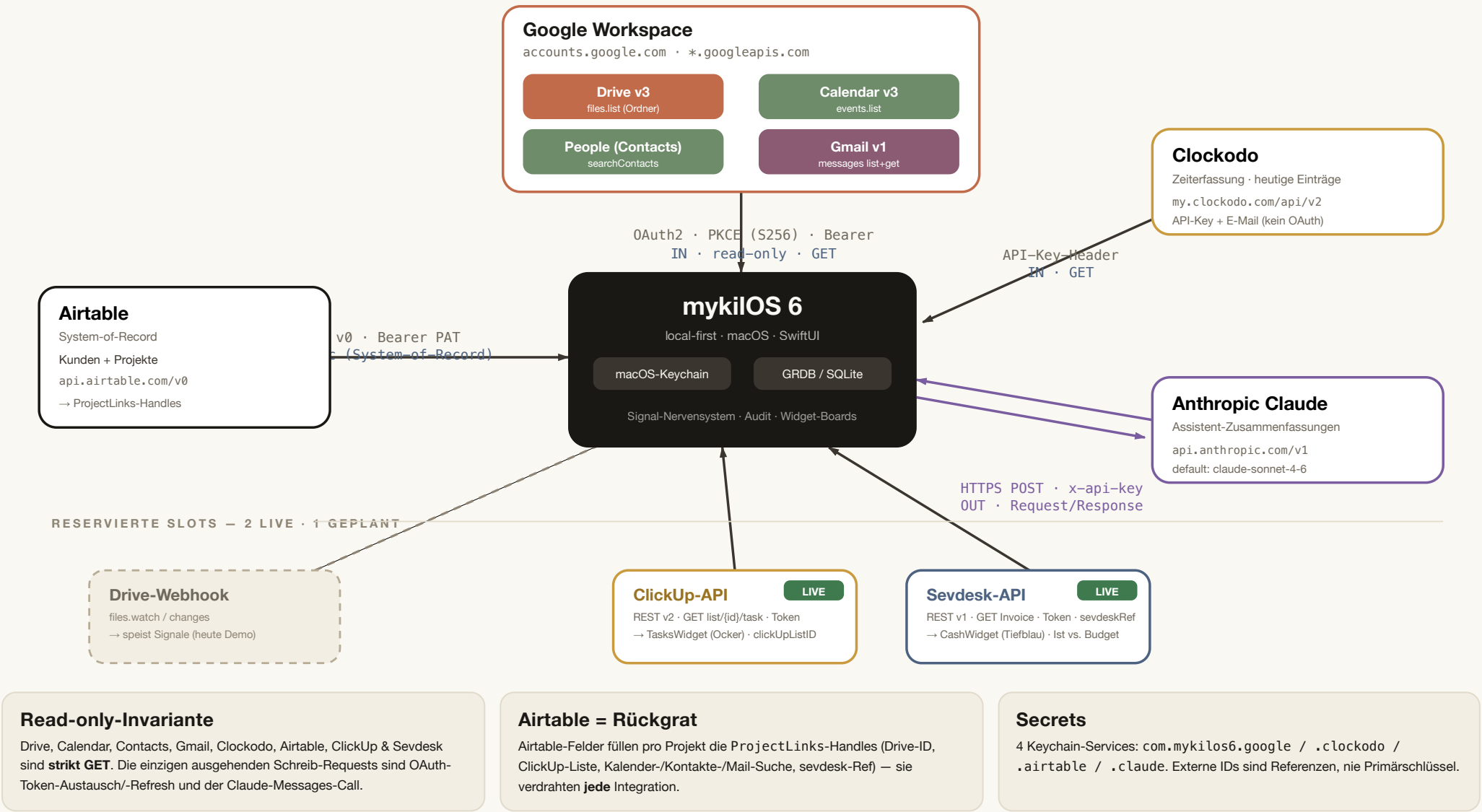
 Demo-Daten, Slot reserviert
- GEPLANT

 noch nicht gebaut
- IN

 liest von extern
- OUT

 sendet nach extern

— durchgezogen = live - - - - - gestrichelt = geplant



Authentifizierung über **OAuth2 mit PKCE** (S256) und lokalem Loopback-Redirect (NWListener auf 127.0.0.1:{dyn. Port}). Tokens liegen in der Keychain; alle vier Clients holen ihr Access-Token zentral über `GoogleAccessTokenProvider`, der bei Ablauf automatisch refresht.

Externe Endpunkte

ZWECK	METHODE	URL	AUTH	RICHTUNG	PARAMETER / NOTIZ
Authorize	GET	accounts.google.com/o/oauth2/v2/auth	PKCE challenge + state	OUT	client_id, scope, code_challenge=S256, access_type=offline, prompt=consent
Token / Refresh	POST	oauth2.googleapis.com/token	form: client_id + code/refresh_token	OUT	grant_type=authorization_code refresh_token
Drive	GET	www.googleapis.com/drive/v3/files	Bearer	IN	q='{folderID}' in parents, fields, pageSize, orderBy
Calendar	GET	www.googleapis.com/calendar/v3/calendars/primary/events	Bearer	IN	timeMin/timeMax (14 T.), q, singleEvents, orderBy
Contacts	GET	people.googleapis.com/v1/people:searchContacts	Bearer	IN	query, readMask=names,emailAddresses,phoneNumbers,organizations
Gmail (Liste)	GET	gmail.googleapis.com/gmail/v1/users/me/messages	Bearer	IN	q (Gmail-Suchsyntax), maxResults=10 → IDs
Gmail (Detail)	GET	.../messages/{id}	Bearer	IN	format=metadata, metadataHeaders=Subject,From,Date (N+1 pro ID)

OAuth-Scopes (verbatim)

- auth/drive.metadata.readonly
- auth/calendar.events.readonly
- auth/gmail.readonly
- auth/contacts.readonly

gmail.readonly (statt .metadata), weil Volltextsuche es verlangt.

Trigger & Handshake

- Verbinden** → Browser → Loopback-Redirect mit ?code&state → Token-Exchange
- Widget lädt** → validAccessToken() → ggf. Refresh → Bearer-GET
- Token-Ablauf** (expiresAt ≤ jetzt) → automatischer Refresh
- Trennen** → Keychain leeren

Secret & Handles

Keychain: com.mykilos6.google
accounts: tokens (JSON), clientID

Handles: driveFolderID, calendarQuery, contactsQuery, mailQuery (aus ProjectLinks)

Bekannter Punkt

Ein fehlgeschlagener Refresh (widerrufenenes Refresh-Token) landet aktuell als generischer `.error("httpError(...)")` im Widget — nicht als `.permissionRequired` mit „Bitte neu verbinden“. Bewusst einfach für V1. Reine Parser-Bausteine (`buildListURL`, `parseFiles` ...) sind vollständig unit-getestet ohne Netz/Keychain.

Clockodo — Zeiterfassung	
Holt die heutigen Zeiteinträge read-only. Auth per Header (API-Key + E-Mail), kein OAuth. Keine Pagination.	
Endpunkt	GET my.clockodo.com/api/v2/entries
Query	time_since / time_until (Tagesbereich, ISO8601)
Auth-Header	X-ClockodoApiUser · X-ClockodoApiKey · X-Clockodo-External-Application
Keychain	com.mykilos6.clockodo accounts: email, apiKey
Liefert	ClockodoTimeEntry: id, label, durationSeconds Label-Fallback: Projekt → Kunde → „(ohne Projekt)“
Status	disconnected · connected · error(String)
Richtung	IN read-only

Airtable — System-of-Record	
Wahrheitsquelle für Kunden & Projekte. Lädt zwei Tabellen, mappt auf Domain-Objekte, persistiert lokal (FileBackedRepository) für Offline/Restart.	
Endpunkt	GET api.airtable.com/v0/{baseID}/{table}
Tabellen	Kunden · Projekte
Auth	Authorization: Bearer {PAT}
Pagination	offset · pageSize=100
Keychain	com.mykilos6.airtable accounts: pat, baseID
Status	disconnected · connected · syncing · error(String)
Richtung	IN read-only (kein Write-back)

Feld-Mapping: Airtable → ProjectLinks-Handles (das Rückgrat)		
AirtableClient.mapProjects übersetzt Airtable-Spalten in die externen Referenz-Handles jedes Projekts — diese füttern danach jedes Widget:		
AIRTABLE-FELD	→ PROJECTLINKS	NUTZT INTEGRATION
Drive-Ordner-ID	driveFolderID	 Google Drive
Kalender-Suche	calendarQuery	 Google Calendar
Kontakte-Suche	contactsQuery	 Google People
Mail-Suche	mailQuery	 Gmail
ClickUp-Liste	clickUpListID	 ClickUp LIVE
sevdesk-Ref	sevdeskRef	 Sevdesk LIVE
Budget	budget	 Soll-Wert für Cash-Balken LIVE
Projektnummer · Titel · Art · Kundennummer · Phase · Eltern-Projekt	Project.* Felder	Domain-Identität
Auto-Sync-Trigger: AppState.bootstrap() ruft bei status == .connected automatisch syncFromAirtable(baseID); zusätzlich manuell über „Jetzt synchronisieren“ in den Einstellungen. Die airtableRecordID bleibt als Rückverweis am Domain-Objekt.		

Anthropic Claude, ClickUp, Sevdesk & geplanter Anschluss

Anthropic Claude — Assistent-Zusammenfassungen LIVE

Der Assistent zeigt sofort regelbasierte Insights und lädt — nur bei verbundener Claude-Konfiguration — zusätzlich eine natürliche Zusammenfassung. **Claude schreibt nie**; Aktionen bleiben bestätigungspflichtig.

Endpunkt	POST <code>api.anthropic.com/v1/messages</code>
Header	<code>x-api-key</code> · <code>anthropic-version: 2023-06-01</code> · <code>Content-Type: application/json</code>
Modell	<code>default claude-sonnet-4-6</code> · <code>max_tokens 420</code>
Payload	System-Prompt (DE) + User-Prompt aus Signalen & Insights als Bulletlisten
Keychain	<code>com.mykilos6.claude</code> <code>accounts: apiKey, model</code>
Trigger	<code>.task(id: hash(signals))</code>
Richtung	OUT Request/Response

Write-Guard: Die LLM-Antwort erzeugt nie einen Audit-Eintrag. Erst der manuelle „Bestätigen“-Tap schreibt eine `AuditEntry`.

ClickUp — Aufgaben LIVE

Slot: `WidgetKind .tasks` (Ocker) · `TasksWidget` liest die offenen Aufgaben der verlinkten Liste, mit allen `Renderstates` (`loading/content/empty/permissionRequired/error`).

Endpunkt: GET `api.clickup.com/api/v2/list/{id}/task` (`archived=false`, `include_closed=false`), Token im `Authorization-Header`.

Handle: `clickUpListID` (in `Airtable` gemappt) · **Keychain:** `com.mykilos6.clickup`. `Read-only`; `Write-back` (Task erledigt) bliebe `Action-Card` → `Bestätigung` → `Audit`.

Sevdesk — Umsatz LIVE

Slot: `WidgetKind .cash` (Tiefblau) · `CashWidget` zeigt den Ist-Umsatz (Summe der Rechnungen für `sevdeskRef`) gegen das Soll-Budget aus `Airtable` als Balken — plus den `Drive→Cash`-Signal-Empfang.

Endpunkt: GET `my.sevdesk.de/api/v1/Invoice` (`contact[id]=ref`, `limit=100`), Token im `Authorization-Header`.

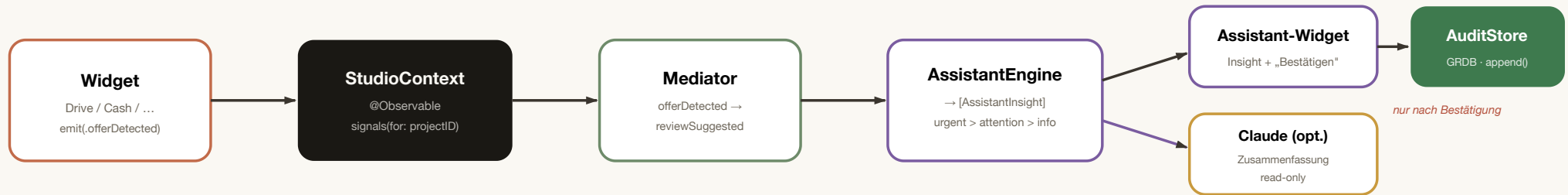
Handle: `sevdeskRef` + `budget` (beide in `Airtable` gemappt) · **Keychain:** `com.mykilos6.sevdesk`. `Read-only`; `mykilOS` bucht nie nach `sevdesk`.

Drive-Webhook GEPLANT

Echtes `Drive-Änderungs-Event` (`files.watch/changes`) soll künftig das Signal `offerDetected` feuern (Drive findet Rechnungs-PDF → Cash fragt). Heute wird dieses Signal nur vom `Demo-Button` erzeugt; im lokalen Desktop-Kontext vermutlich als `Polling` von `changes.list`.

Signal-Nervensystem & Audit

Widgets senden typisierte Ereignisse an den gemeinsamen StudioContext. Der Mediator leitet (azyklisch) Folgesignale ab. Die AssistantEngine formt daraus priorisierte Insights. **Signale sind Vorschläge** — der einzige Schreibvorgang ist der bestätigte Audit-Eintrag.



WidgetSignal (6)

- projectFocused
- driveFileAdded
- offerDetected
- reviewSuggested (Vorschlag)
- budgetThresholdCrossed
- deadlineNear

WidgetSource (8) · Farbe = Sprache



AuditEntry.Action (5) — einzige Writes

- offerImported
- draftCreated
- draftSent
- projectLinked
- noteUpdated

Felder: id · timestamp · actorUserID · projectID · action · summary

Invariante

Mediator-Regeln sind azyklisch (abgeleitete Signale lösen keine weitere Ableitung aus). AssistantEngine ist synchron & testbar; Netzwerk passiert außerhalb. Heute feuert `offerDetected` nur per Demo-Button — die Live-Quelle (Drive-Webhook) ist geplant.

Persistenz & Cold-Start

GRDB-Datenbank

Datei: ~/Library/Application Support/mykil056/db.sqlite · WAL · serialisierte Writes

TABELLE	INHALT / SCHLÜSSEL	STORE
widgetInstances	Layout je boardID (kind, size, position, sichtbar)	WidgetBoardStore
notes	eine Notiz je boardID (INSERT OR REPLACE)	NoteStore
auditEntries	Audit-Log je projectID (append, DESC)	AuditStore

Migration v1_widgets_notes aktiv. Migrationen nur anhängen, nie ändern. AppDatabase.production nutzt try! (hartes Scheitern bei fehlendem Application Support — bewusst).

FileBackedRepository (JSON)

Disk-Cache für customers & projects (aus Airtable). Atomare Writes (temp + rename).

Caveat: Date-Kodierung zwingend über `timeIntervalSinceReferenceDate` — nur das ist bitgenau roundtrip-sicher (ISO8601/secondsSince1970 verlieren Präzision).

SaveState — sichtbarer Speicher-Vertrag

idle saving saved(Date) failed(String)

Jeder Schreibvorgang throws — kein stilles Scheitern. Die `SaveStateBar` zeigt den Zustand im Board direkt an und bietet manuelles Speichern.

Cold-Start-Vertrag

- **Leeres Board** → `load()` pflanzt das Default-Layout je `ProjectKind` und speichert sofort.
- **Leere Registry** → `seedIfEmpty()` injiziert `DemoSeed` (4 Kunden, 6 Projekte).
- **Test je Feature:** schreiben → neue Instanz → lesen → identisch.

Secrets-Trennung

Tokens/Keys/PATs liegen **ausschließlich** in der Keychain (4 Services) — nie in DB, JSON, Code oder Logs. Die DB enthält nur Layout, Notizen und Audit; der JSON-Cache nur Referenz-IDs.

App-Struktur & Widgets

Navigationsbaum

```
MykilOS6App (@main)
├─ AppState @MainActor @Observable
│   └─ GRDBDatabase · RegistryStore
│       └─ homeBoard · homeNotes · audit
│           └─ Auth: google · clockodo · clickup · sevdesk · airtable · claude
├─ StudioContext (Signale)
├─ Fenster: Haupt (1340×860) · „Über“ (440×300)
└─ Sidebar / AppModule
    ├─ Heute → TodayView → HomeBoardView
    ├─ Projekte → ProjectGalleryView → ProjectDetailView
    ├─ Assistant → AssistantPageView
    ├─ Marken & Daten GEPLANT
    ├─ Angebote GEPLANT
    └─ Einstellungen → SettingsView (6 Sektionen)
```

Bootstrap: homeBoard/notes/audit laden → registry seed+load → bei verbundenem Airtable Auto-Sync.

Heute-Board & Projekt-Board

```
HomeBoardView (3-Spalten-Grid, Drag&Drop)
  focus · projectFaves · clockodo
  recentActivity · notes

ProjectDetailView (Tabs, Drag&Drop)
  Hero + ProjectLinks → Widgets
  Tabs: Übersicht (live) · Dateien/
  Angebote/Timeline/Material GEPLANT

Drag&Drop: jede Kachel ist draggable (UUID) → handleDrop() → boardStore.move() → SaveStateBar.
Projekt öffnen: context.focus() + Board/Notes laden; ProjectLinks (driveFolderID, calendar/contacts/mailQuery) gehen
an die Widgets.
```

Widget-Katalog (12 Kinds)

WIDGET	QUELLE / FARBE	DATENQUELLE	STATUS	RENDER-STATES	HANDLE / TRIGGER
Drive	drive · Terrakotta	GoogleDriveClient (files.list)	LIVE	alle 6	driveFolderID · .task
Calendar	calendar · Salbei	GoogleCalendarClient (14 T.)	LIVE	alle 6	calendarQuery
Contacts	people · Salbei	GoogleContactsClient (People)	LIVE	alle 6	contactsQuery
Mail	mail · Pflaume	GoogleGmailClient (list+get)	LIVE	alle 6	mailQuery
Notes	notes · Pflaume	NoteStore (GRDB, lokal)	LIVE	content + Autosave 0,8 s	boardID
Assistant	assistant · Tinte	AssistantEngine (+ Claude opt.)	LIVE	content	signals(for:)
Tasks	tasks · Ocker	ClickUp offene Aufgaben	LIVE	alle States	clickUpListID
Cash	cash · Tiefblau	Sevdesk Ist-Umsatz vs. Budget + Signal	LIVE	content + loading/empty/perm/error inline	sevdeskRef · budget

Heute-spezifisch: focus · projectFaves · clockodo · recentActivity (Clockodo-Widget live über ClockodoClient).

Trigger-, Handshake- & Handle-Matrix

Trigger & Effekte

TRIGGER	EFFEKT	PROTOKOLL
App-Start (bootstrap)	GRDB laden → Registry seed+load → bei verbundenem Airtable: Auto-Sync mit gespeicherter baseID	lokal + REST
„Verbinden" Google	Loopback-Server → Browser → Redirect ?code&state → Token-Exchange → Keychain	OAuth2 PKCE
Widget erscheint	.task → validAccessToken (ggf. Refresh) → Bearer-GET → Render-State	HTTPS GET
Token abgelaufen	GoogleAccessTokenProvider → Refresh-POST → neue Tokens in Keychain	OAuth2 refresh
„Jetzt synchronisieren" Airtable	status .syncing → fetchRecords (paginiert) → map → CachedProjectRegistry → Disk → .connected	REST Bearer PAT
Clockodo/Claude verbinden	connect(...) → Keychain → status .connected (Validierung)	API-Key / x-api-key
Kachel ziehen (Board)	draggable UUID → handleDrop → boardStore.move → SaveStateBar zeigt Speicherzustand	lokal (GRDB)
Notiz tippen	0,8 s Debounce → NoteStore.save (throws) → SaveState	lokal (GRDB)
Signal-Demo-Button	emit(offerDetected/budgetThresholdCrossed/deadlineNear) → Mediator → AssistantEngine	intern
„Bestätigen" am Insight	AuditEntry → AuditStore.append → GRDB (der einzige Write des Assistenten)	lokal (GRDB)
Assistent lädt (Signale ändern sich)	.task(id: hash(signals)) → Claude-POST → Zusammenfassung (read-only)	HTTPS POST

Referenz-Handles (ProjectLinks) — nie Primärschlüssel

driveFolderID	Google-Drive-Ordner
driveFolderPath	lesbarer Pfad
calendarQuery	Kalender-Suche
contactsQuery	Kontakte-Suche
mailQuery	Gmail-Suche/Label
clickUpListID	ClickUp-Liste LIVE
sevdeskRef	Sevdesk-Kontakt-ID LIVE
budget	Soll-Budget (Cash-Balken) LIVE

Geschäftsschlüssel & Secrets

projectNumber	Projekt-Business-Key (z. B. ME-24)
customerNumber	Kunden-Key (z. B. K-1001)
parentProjectNumber	Nachtrag → Eltern-Projekt
airtableRecordID	Rückverweis zur Wahrheitsquelle
boardID	home project({nr}) → Layout-Key

Keychain-Services: com.mykilos6.google · .clockodo · .airtable · .clickup · .sevdesk · .claudе.